Machine Learning Optimization: Theory and Practice

Multiple Authors

May 14, 2025

Contents

1 Introduction to the course. Function classes for optimization. 1

1.1 Introduction . . . 1
1.2 Function Classes for Optimization . . . 1
1.2.1 Convex Functions . . . 1
1.2.2 Smooth Functions . . . 2
1.2.3 Strongly Convex Functions . . . 3
1.2.4 Lipschitz Continuous Functions . . . 3
1.3 Finding Hessians using Second Differentials . . . 3
1.3.1 Second Differentials and the Hessian Matrix . . . 3
1.3.2 Computing Hessians in Practice . . . 3
1.4 Convergence Speed of Optimization Algorithms . . . 3
1.4.1 Rates of Convergence . . . 3
1.4.2 Linear vs. Superlinear vs. Quadratic Convergence . . . 3
1.4.3 Convergence Analysis Framework . . . 3
1.5 Non-exact One-dimensional Optimization . . . 3
1.5.1 Line Search Methods . . . 3
1.5.2 Bracketing Methods . . . 3
1.5.3 Interpolation Methods . . . 3
1.6 Applications in Machine Learning . . . 3
1.7 Conclusion . . . 3

2 Gradient Descent. Convergence speed for different function classes. Examples. Finite-difference methods. 5

2.1 Introduction . . . 5
2.2 The Gradient Descent Algorithm . . . 5
2.3 Convergence Speed for Different Function Classes . . . 8
2.3.1 Convex Functions . . . 8
2.3.2 Strongly Convex Functions . . . 8
2.3.3 Non-convex Functions . . . 8
2.4 Variants of Gradient Descent . . . 8
2.4.1 Batch Gradient Descent . . . 8
2.4.2 Stochastic Gradient Descent . . . 8
2.4.3 Mini-batch Gradient Descent . . . 8
2.5 Finite-difference Methods . . . 8
2.5.1 Forward Difference . . . 8
2.5.2 Central Difference . . . 8
2.5.3 Higher-order Differences . . . 8
2.6 Applications in Machine Learning . . . 8

2.7 Conclusion 8

3 Newton method. Invariance to linear scaling. Global and local convergence. Matrix decompositions for solving linear systems. Hessian correction schemes. 9

4 Conjugate Gradient method for solving linear systems, its convergence properties. Preconditioning. Non-linear CG. 11

5 Hessian-Free Newton (Newton-cg), its convergence speed. Convex sets. Introduction to Quasi-Newton methods. 13

6 Quasi-Newton methods SR1, BFGS and L-BFGS. Convex functions. 15

7 Constrained optimization problem, KKT theorem (first and second order necessary and sufficient conditions). Examples of analytical solving for constrained optimization, DeepFool adversarial attack. 17

8 Dual optimization problem, dual decomposition, duality for Wasserstein distance. Epigraph and reparameterization reformulations. Standard classes of convex constrained optimization problems. 19

9 Revised simplex method for Linear Programming. Log-Barrier method for constrained convex programming. 21

10 Non-smooth convex optimization: examples, subgradient method. Subdifferential calculus. 23

11 Proximal Gradient Method. Closed functions, Fenchel conjugate functions and conjugate (dual) norms 25

12 Projections and proximal operators. Differentiation through optimization methods. 27

13 Stochastic optimization. 29

Preface

This book covers various optimization methods used in machine learning, from fundamental algorithms to advanced techniques. Each chapter focuses on specific optimization methods and their applications in machine learning.

Chapter 1

Introduction to the course. Function classes for optimization.

Abstract:
This chapter provides an introduction to optimization in the context of machine learning. We discuss various function classes for optimization, methods for finding Hessians using second differentials, convergence speed analysis of optimization algorithms, and non-exact one-dimensional optimization techniques.

1.1 Introduction

Optimization is a central problem in machine learning, forming the foundation of model training and parameter estimation. This chapter establishes key concepts, terminology, and mathematical foundations that will be used throughout this book.

1.2 Function Classes for Optimization

1.2.1 Convex Functions

A function f is convex if its domain is a convex set and if for all x, y in the domain and $\alpha \in [0, 1]$:

Definition 1.1 (Convexity)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if for all $x, y \in \mathrm{dom}(f)$ and $\alpha \in [0, 1]$:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$$

Theorem 1.1 (First-order Convexity Condition)**.** For a differentiable function f, it is convex if and only if its domain is convex and

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

for all $x, y \in \mathrm{dom}(f)$.

Proof: Let's consider the function $g(t) = f(x + t(y - x))$ for $t \in [0, 1]$. By the convexity of f, we have $g(t) \leq (1 - t)g(0) + tg(1)$ which gives:

$$f(x + t(y - x)) \leq (1 - t)f(x) + tf(y)$$

Taking the derivative of $g(t)$ at $t = 0$:

$$g'(0) = \nabla f(x)^T (y - x)$$

By convexity, $g(1) - g(0) \geq g'(0)(1 - 0)$, which gives:

$$f(y) - f(x) \geq \nabla f(x)^T (y - x)$$

Rearranging, we get $f(y) \geq f(x) + \nabla f(x)^T (y - x)$. □

1.2.2 Smooth Functions

Smoothness is another key property in optimization.

Definition 1.2 (L-smoothness)**.** A differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ is L-smooth if its gradient is L-Lipschitz continuous, i.e., for all $x, y \in \mathrm{dom}(f)$:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

Claim 1.1 (Descent Lemma)**.** If f is L-smooth, then for all $x, y \in \mathrm{dom}(f)$:

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{L}{2}\|y - x\|^2$$

Lemma 1.1 (Quadratic Upper Bound)**.** For an L-smooth function f, the function value can be upper-bounded by a quadratic approximation:

$$f(x + d) \leq f(x) + \nabla f(x)^T d + \frac{L}{2}\|d\|^2$$

Example 1.1 (Smooth Function)**.** The function $f(x) = \frac{1}{2}x^T Ax + b^T x + c$, where A is positive definite, is smooth with constant $L = \lambda_{\max}(A)$, the largest eigenvalue of A.

1.2.3 Strongly Convex Functions

1.2.4 Lipschitz Continuous Functions

1.3 Finding Hessians using Second Differentials

1.3.1 Second Differentials and the Hessian Matrix

1.3.2 Computing Hessians in Practice

1.4 Convergence Speed of Optimization Algorithms

1.4.1 Rates of Convergence

1.4.2 Linear vs. Superlinear vs. Quadratic Convergence

1.4.3 Convergence Analysis Framework

1.5 Non-exact One-dimensional Optimization

1.5.1 Line Search Methods

Line search is a fundamental technique for determining step sizes in optimization algorithms.

Example 1.2 (Backtracking Line Search)**.** Backtracking line search is a practical algorithm for finding an appropriate step size that satisfies the Armijo condition. It is widely used in gradient descent and Newton's method implementations.

Algorithm 1.1 Backtracking Line Search

Require: Function f, current point x, descent direction d, initial step size $\alpha > 0$, parameters $0 < \beta < 1$ and $0 < \sigma < 0.5$
Ensure: Step size α
  while $f(x + \alpha d) > f(x) + \sigma\alpha\nabla f(x)^T d$ **do**
    $\alpha \leftarrow \beta\alpha$ {Reduce step size}
  end while
  return α

The Armijo condition $f(x + \alpha d) \leq f(x) + \sigma\alpha\nabla f(x)^T d$ ensures sufficient decrease in the function value. Typical values are $\beta = 0.5$ (reducing step size by half each iteration) and $\sigma = 0.1$.

1.5.2 Bracketing Methods

1.5.3 Interpolation Methods

1.6 Applications in Machine Learning

1.7 Conclusion

Chapter 2

Gradient Descent. Convergence speed for different function classes. Examples. Finite-difference methods.

Abstract:
This chapter explores gradient descent methods for optimization in machine learning. We analyze convergence rates for different function classes, provide practical examples, and discuss finite-difference approximations when gradients are not analytically available.

2.1 Introduction

Gradient descent is one of the most fundamental algorithms in optimization, serving as the cornerstone for many machine learning algorithms. This chapter introduces the method, its variants, convergence properties, and practical implementations.

2.2 The Gradient Descent Algorithm

Definition 2.1 (Gradient Descent)**.** Gradient descent is an iterative optimization algorithm that updates parameters in the direction of the negative gradient. Given a function $f : \mathbb{R}^n \to \mathbb{R}$, the update rule is:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

where $\alpha_k > 0$ is the step size or learning rate at iteration k.

Algorithm 2.1 Gradient Descent with Backtracking Line Search

Require: Function f, gradient ∇f, initial point x_0, tolerance $\epsilon > 0$, maximum iterations K
Require: Initial step size $\alpha_0 > 0$, backtracking parameters $0 < \beta < 1$, $0 < \sigma < 0.5$
Ensure: Approximate minimizer x^*

```
k ← 0
while ‖∇f(x_k)‖ > ε and k < K do
    d_k ← −∇f(x_k) {Descent direction}
    α ← α_0 {Reset step size}
    while f(x_k + αd_k) > f(x_k) + σα∇f(x_k)^T d_k do
        α ← βα {Reduce step size}
    end while
    x_{k+1} ← x_k + αd_k {Update point}
    k ← k + 1
end while
return x_k
```

Theorem 2.1 (Convergence of Gradient Descent for L-smooth Functions)**.** If f is L-smooth and convex, and gradient descent is run with constant step size $\alpha \leq \frac{1}{L}$, then after k iterations:

$$f(x_k) - f(x^*) \leq \frac{\|x_0 - x^*\|^2}{2\alpha k}$$

where x^* is a minimizer of f.

Proof: From L-smoothness, we have:

$$f(x_{k+1}) \leq f(x_k) + \nabla f(x_k)^T (x_{k+1} - x_k) + \frac{L}{2}\|x_{k+1} - x_k\|^2$$

Substituting $x_{k+1} = x_k - \alpha \nabla f(x_k)$:

$$f(x_{k+1}) \leq f(x_k) - \alpha\|\nabla f(x_k)\|^2 + \frac{L\alpha^2}{2}\|\nabla f(x_k)\|^2$$

When $\alpha \leq \frac{1}{L}$, we have $\frac{L\alpha^2}{2} \leq \frac{\alpha}{2}$, so:

$$f(x_{k+1}) \leq f(x_k) - \frac{\alpha}{2}\|\nabla f(x_k)\|^2$$

By convexity of f, we have $f(x_k) - f(x^*) \leq \nabla f(x_k)^T (x_k - x^*)$. Using the Cauchy-Schwarz inequality and some algebraic manipulations (omitted for brevity), we can establish the result. □

Claim 2.1 (Convergence for Strongly Convex Functions)**.** If f is μ-strongly convex and L-smooth, gradient descent with constant step size $\alpha = \frac{1}{L}$ converges linearly:

$$f(x_k) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)^k [f(x_0) - f(x^*)]$$

Example 2.1 (Quadratic Function)**.** Consider minimizing $f(x) = \frac{1}{2}x^T Ax + b^T x + c$ where A is positive definite. The gradient is $\nabla f(x) = Ax + b$, and gradient descent with optimal step size converges linearly with rate determined by the condition number of A.

Lemma 2.1 (Optimal Step Size)**.** For minimizing a quadratic function $f(x) = \frac{1}{2}x^T Ax + b^T x + c$, the optimal fixed step size is $\alpha = \frac{2}{\lambda_min(A)+\lambda_max(A)}$.

Example 2.2 (Python Implementation of Gradient Descent)**.** Below is a pseudocode implementation of gradient descent in Python-like syntax:

Algorithm 2.2 Gradient Descent Implementation

Require: f, ∇f, x_0, α, ϵ, max_iter
  $x \leftarrow x_0$
  $iter \leftarrow 0$
  $grad_norm \leftarrow \|\nabla f(x)\|$
  while $grad_norm > \epsilon$ and $iter < max_iter$ **do**
    $x \leftarrow x - \alpha \cdot \nabla f(x)$
    $grad_norm \leftarrow \|\nabla f(x)\|$
    $iter \leftarrow iter + 1$
    if $iter \bmod 100 = 0$ **then**
      Print "Iteration:", $iter$, "Function value:", $f(x)$, "Gradient norm:", $grad_norm$
    end if
  end while
  return x

For functions with expensive gradient computations, we can save computation by reusing the gradient:

Algorithm 2.3 Memory-Efficient Gradient Descent

Require: f, ∇f, x_0, α, ϵ, max_iter
  $x \leftarrow x_0$
  $iter \leftarrow 0$
  $grad \leftarrow \nabla f(x)$
  $grad_norm \leftarrow \|grad\|$
  while $grad_norm > \epsilon$ and $iter < max_iter$ **do**
    $x \leftarrow x - \alpha \cdot grad$
    $grad \leftarrow \nabla f(x)$
    $grad_norm \leftarrow \|grad\|$
    $iter \leftarrow iter + 1$
  end while
  return x

2.3 Convergence Speed for Different Function Classes

2.3.1 Convex Functions

2.3.2 Strongly Convex Functions

2.3.3 Non-convex Functions

2.4 Variants of Gradient Descent

2.4.1 Batch Gradient Descent

2.4.2 Stochastic Gradient Descent

2.4.3 Mini-batch Gradient Descent

2.5 Finite-difference Methods

2.5.1 Forward Difference

2.5.2 Central Difference

2.5.3 Higher-order Differences

2.6 Applications in Machine Learning

2.7 Conclusion

Chapter 3

Newton method. Invariance to linear scaling. Global and local convergence. Matrix decompositions for solving linear systems. Hessian correction schemes.

Chapter 4

Conjugate Gradient method for solving linear systems, its convergence properties. Preconditioning. Non-linear CG.

Chapter 5

Hessian-Free Newton (Newton-cg), its convergence speed. Convex sets. Introduction to Quasi-Newton methods.

Chapter 6

Quasi-Newton methods SR1, BFGS and L-BFGS. Convex functions.

Chapter 7

Constrained optimization problem, KKT theorem (first and second order necessary and sufficient conditions). Examples of analytical solving for constrained optimization, DeepFool adversarial attack.

Chapter 8

Dual optimization problem, dual decomposition, duality for Wasserstein distance. Epigraph and reparameterization reformulations. Standard classes of convex constrained optimization problems.

Chapter 9

Revised simplex method for Linear Programming. Log-Barrier method for constrained convex programming.

Chapter 10

Non-smooth convex optimization: examples, subgradient method. Subdifferential calculus.

Chapter 11

Proximal Gradient Method. Closed functions, Fenchel conjugate functions and conjugate (dual) norms

Chapter 12

Projections and proximal operators. Differentiation through optimization methods.

Chapter 13

Stochastic optimization.